Prompt:
LLM Task: Code Divergence Analysis for Functional Equivalence

You are an expert software engineer and code analysis tool. Your task is to perform a deep, comparative analysis between two programs written in different languages (Java and Rust) that are intended to perform the exact same task. The goal is to identify all implementation differences that could potentially lead to divergent *program behavior* or *output* when given the same input.
---
1. Input Files
File A: Java Implementation (Reference)
```
public void clear() {
Logger.debug("Schedule:" + name + ". clear() called");
for (Slot curSlot : slotVector) {
curSlot.reset();
}
acceptedReservations.clear();
}
public LoadStatus getSimulationLoad() {
return this.loadBuffer.getCuttedOverallLoad();
}
public Reservation reserve(Reservation reservation) {
// // catch reservation was already reserved for this schedule
// if (acceptedReservations.containsSameName(reservation)) {
// Logger.fatal("Schedule:" + name + ". " + "id not unique: "
// + reservation.getJobName());
// }
// bring scheduling window up to date
update();
// calculate reservation candidates
// normally the booking interval of a reservation will be the same with the
// assigned start and end, since it was adjusted by a probe call before.
// So in normal case we will get only one candidate
Reservations searchResults = calculateSchedule(reservation);
// catch no possible reservations found (something have changed since the probe
request)
if (searchResults.isEmpty()) {
reservation.setState(ReservationState.REJECTED);
return reservation;
```

```
}
// we only expect one candidate
Reservation curRes = searchResults.firstElement();
this.isFragCacheUpToDate = false;
return reserveWithoutCheck(curRes);
}
public double getFragmentation(long fragStartTime, long fragEndTime) {
// bring scheduling window up to date
update();
// catch wrong times
if (fragEndTime == Simulator.TIME_NOT_SET) {
fragEndTime = Simulator.TIME_INFINITY;
}
else if (fragEndTime <= fragStartTime) {
Logger.fatal("Schedule:" + name + ". " + "endTime (" + fragEndTime
+ ") <= startTime (" + fragStartTime + "). Error");
}
// compute start/end slot indexes
long startSlotIndex = this.getSlotIndex(fragStartTime);
startSlotIndex = this.cutSlotIndex(startSlotIndex);
long endSlotIndex = this.getSlotIndex(fragEndTime);
endSlotIndex = this.cutSlotIndex(endSlotIndex);
if (quadraticMeanFragmentationCalculation) {
return getFragmentationQuadraticMean(startSlotIndex, endSlotIndex);
} else {
return getFragmentationResubmit(startSlotIndex, endSlotIndex);
}
}
/**
* moves the area of time where something can be scheduled, so that the area begins
with
* the actual simulation time.
* schedule entries before the actual simulation time will be deleted.
*/
@Override
public void update() {
long currentTime = Simulator.getCurrentTimeSeconds();
long newStartSlotIndex = getSlotIndex(currentTime);
if(this.startSlotIndex < newStartSlotIndex)
```

```
this.isFragCacheUpToDate = false;
// clean slots which are earlier then the new start slot
for(long cleanIndex = this.startSlotIndex; cleanIndex < newStartSlotIndex; cleanIndex
++){
Slot nextSlot = this.getSlot(cleanIndex);
if(null != nextSlot){
// remove reservation which end earlier than the new start time
for ( Reservation reservationInSlot : nextSlot.getReservations() ) {
long lastSlotOfReservation = getSlotIndex(reservationInSlot.getAssignedEnd());
if(lastSlotOfReservation == cleanIndex){
acceptedReservations.removeSameName(reservationInSlot);
}
}
// update load statistic
//reservedNoOfSlots += nextSlot.getLoad();
this.loadBuffer.add(nextSlot.getLoad(), cleanIndex);
// clean slot
nextSlot.reset();
}
}
// set new Pointer to start and end of the new scheduling window
this.startSlotIndex = newStartSlotIndex;
this.endSlotIndex = newStartSlotIndex + this.slotVector.size() -1;
// set corresponding time borders for the scheduling window
this.schedulingWindowStartTime = this.getSlotStartTime(this.startSlotIndex);
this.schedulingWindowEndTime = this.getSlotEndTime(this.endSlotIndex);
}
public Reservation deleteReservation(Reservation reservation) {
// catch can not delete unreserved reservation
if (false == acceptedReservations.containsSameName(reservation)) {
Logger.fatal("Schedule:" + name + ". " + "id unknown: "
+ reservation + ". Known ids: " + acceptedReservations);
reservation.setState(ReservationState.REJECTED);
return reservation;
}
// bring scheduling window up to date
update();
// can not delete already finished reservations
boolean jobFinished = reservation.getAssignedEnd() <=
```

```
Simulator.getCurrentTimeSeconds();
if(jobFinished){
Logger.fatal("Schedule:" + name + ". " + "can not delete already finished reservatoion: "
+ reservation);
reservation.setState(ReservationState.REJECTED);
return reservation;
}
// delete reservation from schedule
acceptedReservations.removeSameName(reservation);
// delete reservation from all occupied slots
long resStartSlotIndex = this.getSlotIndex(reservation.getAssignedStart());
long resEndSlotIndex = this.getSlotIndex(reservation.getAssignedEnd());
// delete only parts that are in the scheduling window
if(resStartSlotIndex < this.startSlotIndex)
resStartSlotIndex = this.startSlotIndex;
for (long slotIndex = resStartSlotIndex; slotIndex <= resEndSlotIndex; ++slotIndex) {
Slot slot = this.getSlot(slotIndex);
slot.deleteReservation(reservation);
}
acceptedReservations.removeSameName(reservation);
// update fragmentation cache
// updateFragmentationCache(startSlotNr,endSlotNr);
reservation.setState(ReservationState.DELETED);
this.isFragCacheUpToDate = false;
return reservation;
}
public LoadStatus getLoad(long startTime, long endTime) {
update();
LoadStatus result = new LoadStatus();
// catch wrong times
if (endTime == Simulator.TIME_NOT_SET) {
endTime = Simulator.TIME_INFINITY;
}
else if (endTime < startTime) {
Logger.fatal("Schedule:" + name + ". " + "endTime (" + endTime
+ ") < startTime (" + startTime + "). Error");
}
// get start and end index in schedule
long startSlotNr = this.getSlotIndex(startTime);
```

```
startSlotNr = this.cutSlotIndex(startSlotNr);
long endSlotNr = this.getSlotIndex(endTime);
endSlotNr = this.cutSlotIndex(endSlotNr);
// sum reserved capacity in time interval
double reservedCapacitySum=0;
for (long index = startSlotNr; index <= endSlotNr; index++) {
int realSlotIndex = this.getRealSlotIndex(startSlotNr);
reservedCapacitySum += this.slotVector.get(realSlotIndex).getLoad();
}
// log considered time interval
result.startTime = startTime;
result.endTime = endTime;
// log load
long numberOfSlots = endSlotNr - startSlotNr;
// load is 1 if time interval is 0
double averageReservedCapacity = this.getCapacity();
if(numberOfSlots != 0)
averageReservedCapacity = ((double)reservedCapacitySum) /
((double)(numberOfSlots));
result.averageReservedCapacity = averageReservedCapacity;
result.possibleCapacity = this.getCapacity();
result.utilization = result.averageReservedCapacity / result.possibleCapacity;
//double result = (double) load / (double) ((endSlotNr-startSlotNr+1) * getCapacity());
// Logger.debug("Schedule(" + name +
// ").getAvgLoad()="+load+"/"+nrOfSlotsToUse+"="+result);
return result;
}
public double getSystemFragmentation() {
if(!this.isFragCacheUpToDate) {
this.fragmentationCache = this.getFragmentation(this.schedulingWindowStartTime,
this.schedulingWindowEndTime);
this.isFragCacheUpToDate = true;
}
return fragmentationCache;
}
public Reservations probe(Reservation reservation) {
// bring scheduling window up to date
update();
// calculate and return reservation candidates
```

```
Reservations searchResults = calculateSchedule(reservation);
// calculate fragmentationDelta for every result if requested
// temporary reserve every reservation and calculate the fragmentation before and after
the reservation
//FIXME: isFragNeeded
if(isFragNeeded){
double fragBefore = this.getSystemFragmentation();
for (Reservation nextResult : searchResults) {
Reservation reserveAnswer = this.reserve(nextResult);
double fragAfter = this.getSystemFragmentation();
nextResult.fragDelta = fragAfter - fragBefore;
this.deleteReservation(reserveAnswer);
}
}
return searchResults;
}
public Reservation probeBest( Reservation request,
Comparator<Reservation> comparator) {
Reservations possibleReservations = this.probe(request);
// find best candidate
Reservation bestCandidate = null;
for (Reservation nextReservation : possibleReservations) {
if(bestCandidate == null){
bestCandidate = nextReservation;
}else if(comparator.compare(bestCandidate, nextReservation) > 0)
bestCandidate = nextReservation;
}
return bestCandidate;
}
/**
* does a reservation without prior check if there is enough capacity at the resource
* @param res reservation to reserve
* @return Reservation object containing the result of the operation as status
* (success : STATE_RESERVEANSWER, fail: STATE_REJECTED)
*/
public Reservation reserveWithoutCheck(Reservation res) {
// insert reservation into slots
for ( long slotIndex = this.getSlotIndex(res.getAssignedStart());
slotIndex <= this.getSlotIndex(res.getAssignedEnd());
```

```
slotIndex++) {
this.insertReservationInSlot(res, slotIndex);
//int realSlotIndex = this.getRealSlotIndex(slotIndex);
//this.slotVector.get(realSlotIndex).insertReservation(res.getReservedCapacity(), res);
}
// remember accepted reservations;
acceptedReservations.add(res);
// update statistics
/* if (res.getAssignedStart() < earliestStartEverScheduled)
earliestStartEverScheduled = res.getAssignedStart();
if (latestEndEverScheduled < res.getAssignedEnd())
latestEndEverScheduled = res.getAssignedEnd();
*/
// return success
res.setState(ReservationState.RESERVEANSWER);
return res;
}
**File B: Rust Implementation (Target)**
use std::cmp::Ordering;
use std::fmt::Debug;
use crate::domain::vrm_system_model::reservation::reservation::ReservationKey;
use crate::domain::vrm_system_model::reservation::{reservation::Reservation,
reservations::Reservations};
use crate::domain::vrm_system_model::utils::load_buffer::LoadMetrics;
#[derive(Debug, Clone)]
pub struct SlottedSchedule {
/// **Unique identifier** for this SlottedSchedule.
id: ReservationKey,
/// A list of all time **Slots** defined for this schedule.
slots: Vec<Slot>,
/// The maximum total amount of resource "pieces" (e.g., cores, units) managed by this
schedule.
pub capacity: i64,
/// The duration of a single time slot.
slot_width: i64,
/// The index of the earliest possible slot that can be used for scheduling.
start_slot_index: i64,
/// The index of the latest possible slot that defines the scheduling window's end.
end_slot_index: i64,
```

```
/// The **absolute start time** (e.g., Unix timestamp) of the current scheduling window being viewed.
scheduling_window_start_time: i64,
/// The **absolute end time** (e.g., Unix timestamp) of the current scheduling window being viewed.
scheduling_window_end_time: i64,
/// Internal buffer used to track transient or potential resource load.
load_buffer: LoadBuffer,
/// A map of all currently **active reservations** associated with this schedule.
active_reservations: Reservations,
/// Flag indicating if the stored **fragmentation_cache** value is valid and up-to-date.
is_frag_cache_up_to_date: bool,
/// The cached value of the system **fragmentation**.
fragmentation_cache: f64,
/// A configuration flag to determine if the system should utilize the **quadratic mean**
/// or the standard formula for fragmentation calculation.
use_quadratic_mean_fragmentation: bool,
/// A flag indicating whether fragmentation calculation is required for the **prob requests**.
is_frag_needed: bool,
simulator: Box<dyn SystemSimulator>,
}
// TODO Sync is potentially unsafe; if total struct Sync than this should be redundant
pub trait Schedule: Debug + Send + Sync {
/// Calculates the resource **fragmentation score** over a specific, user-defined time range.
///
/// # Arguments
///
/// * `frag_start_time` - The absolute start time for the fragmentation window (in seconds).
/// * `frag_end_time` - The absolute end time for the fragmentation window (in seconds).
///
/// # Returns
///
/// A `f64` fragmentation score (typically **0.0** being best, **1.0** being worst).
fn get_fragmentation(&mut self, frag_start_time: i64, frag_end_time: i64) -> f64;
/// Calculates the resource **fragmentation score** across the **entire active scheduling window**.
```

```
///
/// This method is intended to provide a system-wide view of allocated slots and often utilizes
/// a cached value for performance if the schedule has not been modified since the last calculation.
fn get_system_fragmentation(&mut self) -> f64;
/// Retrieves resource **load metrics** (e.g., average reserved capacity, utilization)
/// for a specified absolute time interval.
///
/// # Arguments
///
/// * `start_time` - The absolute start time of the query interval (in seconds).
/// * `end_time` - The absolute end time of the query interval (in seconds).
///
/// # Returns
///
/// A `LoadMetrics` structure detailing the average capacity utilization and reserved capacity.
fn get_load_metrics(&mut self, start_time: i64, end_time: i64) -> LoadMetrics;
/// Retrieves load metrics for the **effective overall simulation period**.
///
/// This period excludes initial and final slots defined by system configuration
/// (`SLOTS_TO_DROP_ON_START`/`SLOTS_TO_DROP_ON_END`).
fn get_simulation_load(&mut self) -> LoadMetrics;
/// Performs a **feasibility probe** to find all possible time slots where a given reservation
/// request can be accommodated.
///
/// The probe returns a collection of candidate reservations, each representing a feasible time assignment.
/// Additionally is also the system fragmentation impact of each candidate calculated and in
/// `frag_delta` of the reservation stored.
///
/// # Arguments
///
/// * `key` - The `ReservationKey` identifying the resource requirements and constraints for the probe.
///
```

```
/// # Returns
///
/// A `Reservations` collection containing all feasible candidates.
fn probe(&mut self, key: ReservationKey) -> Reservations;
/// Selects the **single best-fitting reservation candidate** from the feasible set,
/// determined by a custom comparator function.
///
/// This method first performs a standard `probe` to get all candidates, and then uses the provided
/// comparator to select the optimal scheduling choice (e.g., earliest start time, least fragmentation impact).
///
/// # Type Parameters
///
/// * `C` - A closure that implements the `FnMut` trait for comparison, taking two boxed reservations
/// and returning an `Ordering`.
///
/// # Arguments
///
/// * `request_key` - The `ReservationKey` defining the request.
/// * `comparator` - The function used to determine which candidate is "best."
///
/// # Returns
///
/// An `Option` containing the single `Box<dyn Reservation>` that best fits the criteria, or `None` if no feasible candidates were found.
fn probe_best<C>(&mut self, request_key: ReservationKey, comparator: C) -> Option<Box<dyn Reservation>>
where
C: FnMut(Box<dyn Reservation>, Box<dyn Reservation>) -> Ordering;
/// Attempts to execute a **final reservation** using a provided candidate.
///
/// If the attempt succeeds, the capacity is assigned, and `None` is returned. If capacity is
/// unavailable, the reservation is marked as `Rejected` and returned inside `Some`.
///
/// # Arguments
///
```

```
/// * `reservation` - The `Box<dyn Reservation>` candidate to finalize.
///
/// # Returns
///
/// `None` on success (reservation is accepted and committed), or `Some(reservation)` if the reservation is rejected.
fn reserve(&mut self, reservation: Box<dyn Reservation>) -> Option<Box<dyn Reservation>>;
/// **Commits a reservation** to the schedule **without performing a feasibility check**.
///
/// This is an internal function typically called after a successful `probe` or by `reserve`
/// to finalize the assignment. It assumes the reservation details are valid.
///
/// # Arguments
///
/// * `reservation` - The `Box<dyn Reservation>` to be inserted directly into the schedule slots.
fn reserve_without_check(&mut self, reservation: Box<dyn Reservation>);
/// Removes an **active reservation** from the schedule and frees up the occupied capacity
/// in all relevant time slots.
///
/// # Arguments
///
/// * `reservation_key` - The `ReservationKey` of the reservation to be deleted.
fn delete_reservation(&mut self, reservation_key: ReservationKey);
/// **Clears all active reservations** and resets the load of all slots to zero.
fn clear(&mut self);
/// **Updates the scheduling window** by advancing the internal time pointers based on the current simulation time.
///
/// This process deletes all reservations that have expired (assigned end time is past the new start time)
/// and moves the load from the now-expired slots into the `load_buffer` for historical tracking.
fn update(&mut self);
}
use crate::domain::vrm_system_model::reservation::{
reservation::{Reservation, ReservationKey, ReservationState},
```

```
reservations::Reservations,
};
use crate::domain::vrm_system_model::scheduler_trait::Schedule;
use crate::domain::vrm_system_model::utils::load_buffer::{LoadMetrics,
SLOTS_TO_DROP_ON_END, SLOTS_TO_DROP_ON_START};
use std::cmp::Ordering;
use std::collections::HashSet;
use std::i64;
use std::ops::Not;
impl Schedule for super::SlottedSchedule {
fn clear(&mut self) {
log::warn!("In SlottedSchedule id: {}, where all Slots cleared.", self.id);
for slot in self.slots.iter_mut() {
slot.reset();
}
self.active_reservations.clear();
}
fn get_simulation_load(&mut self) -> LoadMetrics {
let index_of_first_slot: i64 = self.load_buffer.context.get_first_load() +
SLOTS_TO_DROP_ON_START;
let start_time_of_first_slot: i64 = self.get_slot_start_time(index_of_first_slot);
let index_of_last_slot: i64 = self.load_buffer.context.get_last_load() -
SLOTS_TO_DROP_ON_END;
let start_time_of_last_slot: i64 = self.get_slot_start_time(index_of_last_slot);
return self.load_buffer.get_effective_overall_load(self.capacity as f64,
start_time_of_first_slot, start_time_of_last_slot);
}
fn reserve(&mut self, mut reservation: Box<dyn Reservation>) -> Option<Box<dyn
Reservation>> {
self.update();
let search_results = self.calculate_schedule(reservation.get_id());
match search_results.get_reservation_with_first_start_slot() {
Some(random_reservation) => {
self.is_frag_cache_up_to_date = false;
self.reserve_without_check(random_reservation.box_clone());
return None;
}
None => {
reservation.set_state(ReservationState::Rejected);
```

```
return Some(reservation);
}
}
}
fn get_fragmentation(&mut self, frag_start_time: i64, frag_end_time: i64) -> f64 {
self.update();
let mut frag_end_time = frag_end_time;
if frag_end_time == i64::MIN {
frag_end_time = i64::MAX
} else if frag_end_time <= frag_start_time {
log::error!(
"Request to get fragmentation of Schedule: {}, the fragmentation start time {} was before
the fragmentation end time {}.",
self.id,
frag_start_time,
frag_end_time,
)
}
let mut start_slot_index = self.get_slot_index(frag_start_time);
start_slot_index = self.get_effective_slot_index(start_slot_index);
let mut end_slot_index = self.get_slot_index(frag_end_time);
end_slot_index = self.get_effective_slot_index(end_slot_index);
if self.use_quadratic_mean_fragmentation {
return self.get_fragmentation_quadratic_mean(start_slot_index, end_slot_index);
}
return self.get_fragmentation_resubmit(start_slot_index, end_slot_index);
}
fn update(&mut self) {
let current_time: i64 = self.simulator.get_current_time_in_s();
let new_start_slot_index = self.get_slot_index(current_time);
if self.start_slot_index < new_start_slot_index {
self.is_frag_cache_up_to_date = false;
}
// key are used to: remove reservation which end earlier than the new start time
let mut keys_to_remove: HashSet<ReservationKey> = HashSet::new();
for clean_index in self.start_slot_index..new_start_slot_index {
if let Some(slot) = self.get_slot(clean_index) {
for key in &slot.reservation_keys {
if let Some(reservation) = self.active_reservations.get(key) {
```

```
let last_slot_of_reservation = self.get_slot_index(reservation.get_assigned_end());
if last_slot_of_reservation == clean_index {
keys_to_remove.insert(key.clone());
}
}
}
}
}
for key in keys_to_remove {
self.active_reservations.delete_reservation(&key);
}
for clean_index in self.start_slot_index..new_start_slot_index {
let load = if let Some(slot) = self.get_slot(clean_index) { slot.load } else { 0 };
self.load_buffer.add(load, clean_index);
if let Some(slot) = self.get_mut_slot(clean_index) {
slot.reset();
} else {
log::error!(
"In SlottedSchedule: {} Happened an error during the update process. Slots are now
invalid due to fail reset of slot {}.",
self.id,
clean_index
)
}
}
// set new Pointer to start and end of the new scheduling window
self.start_slot_index = new_start_slot_index;
self.end_slot_index = new_start_slot_index + (self.slots.len() as i64) - 1;
// set corresponding time borders for the scheduling window
self.scheduling_window_start_time = self.get_slot_start_time(self.start_slot_index);
self.scheduling_window_end_time = self.get_slot_end_time(self.end_slot_index);
}
fn delete_reservation(&mut self, key: ReservationKey) {
// Can not Del unreserved reservation
if !self.active_reservations.contains_key(&key) {
log::error!("DEL Reservation form Schedule: {}, However Schedule does not contain
reservation with id: {}", self.id, key);
self.active_reservations.set_state(&key, ReservationState::Rejected);
return;
```

```
}
// Bring scheduling window up to date
self.update();
// Can not delete already finished reservations
let task_finished: bool = self.active_reservations.get_assigned_end(&key) <=
self.simulator.get_current_time_in_s();
if task_finished {
log::error!("Can't deleted reservation {} form Schedule: {}, because reservation is
already finished.", key, self.id,);
return;
}
// Delete reservation from schedule
let mut reservation = if let Some((_, value)) =
self.active_reservations.delete_reservation(&key) {
value
} else {
log::error!("Del reservation (id: {}) was not possible.", key);
return;
};
// Delete reservation from all occupied slots
let mut reservation_start_slot_index: i64 =
self.get_slot_index(reservation.get_assigned_start());
let reservation_end_slot_index: i64 =
self.get_slot_index(reservation.get_assigned_end());
// Delete only parts that are in the scheduling window
if reservation_start_slot_index < self.start_slot_index {
reservation_start_slot_index = self.start_slot_index;
}
let slotted_schedule_id = self.id.clone();
for slot_index in reservation_start_slot_index..=reservation_end_slot_index {
let slot = self
.get_mut_slot(slot_index)
.expect(&format!("In the SlottedSchedule id: {} was the slot with index: {} not found.",
slotted_schedule_id, slot_index));
slot.delete_reservation(key.clone(), reservation.get_reserved_capacity());
}
reservation.set_state(ReservationState::Deleted);
self.is_frag_cache_up_to_date = false;
return;
```

```
}
fn get_load_metrics(&mut self, start_time: i64, mut end_time: i64) -> LoadMetrics {
self.update();
if end_time == i64::MIN {
end_time = i64::MAX;
}
if end_time < start_time {
log::error!("Start time must be before end time: SlottedSchedule id: {} is end_time: {} <
start_time: {}", self.id, end_time, start_time)
}
let mut start_slot_nr = self.get_slot_index(start_time);
start_slot_nr = self.get_effective_slot_index(start_slot_nr);
let mut end_slot_nr = self.get_slot_index(end_time);
end_slot_nr = self.get_effective_slot_index(end_slot_nr);
let mut reserved_capacity_sum: i64 = 0;
for real_slot_index in start_slot_nr..=end_slot_nr {
// TODO int realSlotIndex = this.getRealSlotIndex(startSlotNr); Bug in original VRM was
fixed
let real_slot_index = self.get_real_slot_index(real_slot_index);
reserved_capacity_sum += self.get_slot_load(real_slot_index);
}
let number_of_slots: i64 = end_slot_nr - start_slot_nr;
if number_of_slots < 0 {
log::error!("The number of slots should never be negative.")
}
let avg_reserved_capacity: f64 =
if number_of_slots != 0 { (reserved_capacity_sum as f64) / (number_of_slots as f64) }
else { self.capacity as f64 };
LoadMetrics {
start_time,
end_time,
avg_reserved_capacity: avg_reserved_capacity,
possible_capacity: self.capacity as f64,
utilization: avg_reserved_capacity / (self.capacity as f64),
}
}
fn get_system_fragmentation(&mut self) -> f64 {
if self.is_frag_cache_up_to_date.not() {
self.fragmentation_cache = self.get_fragmentation(self.scheduling_window_start_time,
```

```
self.scheduling_window_end_time);
self.is_frag_cache_up_to_date = true;
}
return self.fragmentation_cache;
}
fn probe(&mut self, key: ReservationKey) -> Reservations {
self.update();
let mut candidates = self.calculate_schedule(key);
let frag_before: f64 = self.get_system_fragmentation();
if self.is_frag_needed {
for (key, candidate) in candidates.clone().iter() {
let reserve_answer: Option<Box<dyn Reservation>> =
self.reserve(candidate.box_clone());
let frag_delta: f64 = self.get_system_fragmentation() - frag_before;
candidates.set_frag_delta(key, frag_delta);
match reserve_answer {
Some(_) => {}
None => {
self.delete_reservation(candidate.get_id());
}
}
}
}
return candidates;
}
fn probe_best<C>(&mut self, request_key: ReservationKey, mut comparator: C) ->
Option<Box<dyn Reservation>>
where
C: FnMut(Box<dyn Reservation>, Box<dyn Reservation>) -> Ordering,
{
let possible_reservations: Reservations = self.probe(request_key);
if possible_reservations.is_empty() {
return None;
}
let mut best_candidate: Box<dyn Reservation> =
possible_reservations.get_reservation_with_first_start_slot().expect("Error getting
random reservation.").clone();
for (_, candidate) in possible_reservations.iter() {
if comparator(best_candidate.clone(), candidate.clone()) == Ordering::Greater {
```

```
best_candidate = candidate.clone();
}
}
return Some(best_candidate);
}
fn reserve_without_check(&mut self, reservation: Box<dyn Reservation>) {
let new_reservation_key = reservation.get_id();
for slot_index in
self.get_slot_index(reservation.get_assigned_start())..=self.get_slot_index(reservation.g
et_assigned_end()) {
self.insert_reservation_into_slot(&new_reservation_key,
reservation.get_reserved_capacity(), slot_index);
}
self.active_reservations.insert(new_reservation_key.clone(), reservation);
self.active_reservations.set_state(&new_reservation_key,
ReservationState::ReserveAnswer);
}
}
```

Output:
Here is the comparative analysis of the **Java (Reference)** and **Rust (Target)** implementations for the SlottedSchedule class.

This analysis identifies functional divergences, bug fixes, and logic alterations that affect the system's behavior.

Executive Summary

The Rust implementation is largely a faithful port of the Java logic, but it contains **one critical bug fix** regarding load calculation and several deviations in **error handling**. The Rust version is more resilient to runtime errors (logging errors instead of fatal crashes) but introduces a potential risk by continuing execution after detecting invalid time states.

---

1. Critical Logic Divergence: The getLoad Loop Bug

There is a significant functional difference in how the average reserved capacity is calculated. The Java version contains a bug that the Rust version explicitly fixes.

Analysis

- **Java (getLoad):**
  Inside the loop, the code uses startSlotNr (the constant starting index) to fetch the real slot index, instead of the loop variable index.

```
Java
for (long index = startSlotNr; index <= endSlotNr; index++) {
    // BUG: Uses startSlotNr instead of index
    int realSlotIndex = this.getRealSlotIndex(startSlotNr);
    reservedCapacitySum += this.slotVector.get(realSlotIndex).getLoad();
}
```

Result: The Java code sums the load of the **first slot** repeatedly, ignoring all other slots in the interval.

- **Rust (get_load_metrics):**
  The Rust implementation correctly iterates using the loop variable.

```
Rust
for real_slot_index in start_slot_nr..=end_slot_nr {
    // FIXED: Uses the loop variable
    let real_slot_index = self.get_real_slot_index(real_slot_index);
    reserved_capacity_sum += self.get_slot_load(real_slot_index);
}
```

Note: The Rust code actually includes a comment acknowledging this: // Bug in original VRM was fixed.

Impact: The Java version returns incorrect load metrics (utilization stats) for any interval larger than 1 slot. The Rust version returns correct statistics.

---

2. Error Handling and Control Flow

The two implementations differ significantly in how they handle invalid states (e.g., end time appearing before start time).

Feature	**Java Implementation**	**Rust Implementation**	**Impact**
Invalid Time Range (getFragmentation)	Logs Logger.fatal (typically halts/throws).	Logs log::error! but **continues execution**.	**High Risk:** Rust may proceed to calculate indices with invalid times, leading to panic or garbage data later in the method.

Delete Missing Reservation	Logs Logger.fatal, sets state REJECTED, returns.	Logs log::error!, sets state Rejected, returns.	**Medium:** Java treats a missing ID as a fatal system integrity violation; Rust treats it as a recoverable error.
Delete Finished Reservation	Logs Logger.fatal.	Logs log::error!.	**Medium:** Similar to above.

---

3. Arithmetic and Constants

Differences in how time constants and integer limits are handled.

A. Infinite Time Handling

- **Java:** Uses Simulator.TIME_NOT_SET to detect undefined end times, replacing them with Simulator.TIME_INFINITY.
- **Rust:** Uses i64::MIN to detect undefined end times, replacing them with i64::MAX.
- **Divergence:** If Simulator.TIME_NOT_SET in Java is not exactly equivalent to i64::MIN (e.g., if it is -1), there is a risk of input mismatch if these systems communicate via serialized data.

B. Slot Index Calculation

- **Java:** getSimulationLoad delegates entirely to loadBuffer.getCuttedOverallLoad().
- **Rust:** Explicitly calculates the bounds before calling the buffer:
  Rust
  ```
  let index_of_first_slot: i64 = self.load_buffer.context.get_first_load() + SLOTS_TO_DROP_ON_START;
  let index_of_last_slot: i64 = self.load_buffer.context.get_last_load() - SLOTS_TO_DROP_ON_END;
  ```

- **Divergence:** The Rust version explicitly enforces SLOTS_TO_DROP logic within the Schedule class. Unless the Java LoadBuffer class does this internally, the effective simulation window size will differ between the two versions.

---

4. Collection Management and State

How reservations are stored and removed differs, impacting concurrency and safety.

A. Iteration during update()

- **Java:** Iterates over slots and modifies the acceptedReservations collection (via removeSameName) inside the loop.
    - *Risk:* Depending on the underlying collection type of acceptedReservations, this is a classic ConcurrentModificationException risk in Java.
- **Rust:** Uses a two-step process to satisfy the borrow checker.
    1. Collects keys_to_remove into a HashSet.
    2. Iterates the set to remove reservations after the slot loop is finished.
    - *Result:* Safer memory management and iteration stability.

B. The "Double Delete"

- **Java (deleteReservation):** Calls acceptedReservations.removeSameName(reservation) **twice**: once before the slot loop and once after.
- **Rust (delete_reservation):** Removes the reservation from the map **once** (before the slot loop).
- **Divergence:** The Java code appears defensive (trying to ensure removal happens even if logic fails mid-way). The Rust code relies on the Option returned by the map removal to ensure it holds the object before proceeding.

---

5. API and Selection Logic

A. probeBest Selection Strategy

Both implementations search for the "best" candidate (minimizing the comparator), but they initialize the search differently:

- **Java:** Starts with bestCandidate = null. Assigns the first element encountered in the iterator.
- **Rust:** Explicitly retrieves get_reservation_with_first_start_slot() to initialize best_candidate.
- **Divergence:** If the Comparator considers two reservations "equal", Java will return the one that appears first in the standard iterator. Rust will return the one that appears first in the "start slot" order (assuming that's what that specific method returns). If the standard iterator is not sorted by start slot, the tie-breaking behavior differs.

B. probe Loop Efficiency

Both implementations trigger a massive amount of update() calls.

- Inside probe, every candidate is temporarily reserved and then deleted.
- reserve calls update().
- delete calls update().
- **Result:** For N candidates, update() is called $2N + 1$ times. While this is inefficient in

both languages, it ensures they remain synchronized in behavior.

---

6. Summary of Recommendations

To ensure the Rust implementation is functionally equivalent (where desirable) and robust:

1. **Fix Error Control in getFragmentation:** The Rust version logs an error if end_time <= start_time but proceeds to calculate indices. This is dangerous. It should likely return early (e.g., return 0.0 or NaN) to match the "stop the world" effect of the Java Fatal log, or handle it gracefully.
2. **Verify Constants:** Ensure Simulator.TIME_NOT_SET in Java maps to i64::MIN in the data protocol.
3. **Validate Drop Slots:** Confirm that the Java LoadBuffer.getCuttedOverallLoad implementation includes the SLOTS_TO_DROP logic visible in the Rust version. If not, the Rust version is discarding data that Java is keeping.

Next Step: Would you like me to generate a unit test case that specifically targets the getLoad bug to verify the fix in Rust and demonstrate the failure in Java?